

Prompt-Injection Detectors Do Not Transfer, but a LoRA-Tuned LLM Generalizes: A Held-Out Cross-Dataset Study

Anonymous ACL submission

Abstract

Prompt-injection detectors are widely reported at $>99\%$ recall, yet those numbers are almost always measured *in-distribution* on the detector’s own dataset. We ask whether they survive a change of dataset. Evaluating three detectors—a regex pre-filter, a fine-tuned DeBERTa classifier (ProtectAI), and an 8B LLM tuned with a single LoRA adapter—on a strict held-out protocol (train on one public injection set, test on two others never seen in training), we find that reported recall does not transfer: ProtectAI, which reports 99.7% in-distribution, drops to 0.40 recall on a held-out set. A regex catalogue is worse still (0.20). The LoRA-tuned LLM, trained only on the same single set, *generalizes*: it reaches 0.97 ROC-AUC on *both* held-out benchmarks (0.59 / 0.60 recall at 1% FPR), exceeding the fine-tuned classifier on deepset (0.88 / 0.40) and matching its AUC on jailbreak, where reported detectors already do well. The generalization comes with a cost we report honestly: the LLM detector is *more* vulnerable to trivial character perturbations (14% zero-width evasion vs. 0% for the classifier). Our findings argue that detection recall is a dataset-fragile quantity and that no single detector—however strong in-distribution—should be treated as a security guarantee; it is one evadable tier of a defense-in-depth stack.

1 Introduction

Indirect prompt injection—malicious instructions smuggled into the data an LLM or LLM agent consumes—is the top-ranked risk in the 2025 OWASP Top 10 for LLM Applications (OWASP, 2025). A common first line of defense is a *detector*: a classifier that flags injection-bearing inputs before the model acts on them. Commercial and open detectors report striking numbers—ProtectAI’s DeBERTa-v3 injection classifier reports 99.7% recall (Protect AI, 2024), and Meta’s Prompt-Guard reports AUC >0.99 (Meta AI, 2024).

These headline numbers are, almost without exception, measured *in-distribution*: the test set is drawn from the same distribution (often the same corpus) as the training set. Deployment is out-of-distribution by definition—the attacks a detector meets in production are not the ones in its training set. We therefore ask a simple question: *do reported detector numbers survive a change of dataset?*

We evaluate three detectors spanning the practical design space—a hand-written regex pre-filter, a fine-tuned encoder classifier (ProtectAI DeBERTa-v3), and an 8B decoder LLM adapted with a single LoRA (Hu et al., 2022) adapter—under a strict held-out protocol: all detectors are scored on the *identical* rows of two public injection benchmarks that the trained detectors never saw in training. Our contributions:

1. A controlled cross-dataset evaluation showing that reported in-distribution recall does not transfer: the fine-tuned classifier’s recall falls from a reported 99.7% to **0.40** on a held-out set (§4).
2. A LoRA-tuned LLM detector that, trained on a single injection set, *generalizes uniformly* (0.97 AUC on both held-out sets) and exceeds the fine-tuned classifier on the set where the classifier fails to transfer.
3. An honest characterization of the accompanying trade-off: the LLM detector generalizes better but is *more* vulnerable to trivial character-level evasion.

The practical message is not “use an LLM detector”; it is that detection recall is fragile across datasets and evadable under perturbation, so a detector should be one defense-in-depth tier, never the guarantee.

2 Related Work

Prompt-injection detection. Detection-based defenses classify the input text. Open detectors include ProtectAI’s DeBERTa-v3-base classifier (Protect AI, 2024), Meta Prompt-Guard (Meta AI, 2024), and LLM-as-judge preprocessors (pro, 2025). Benchmarks such as PINT (Lakera AI, 2024) and the over-defense benchmark NotInject (inj, 2024) rank detectors, but each detector’s headline recall is reported on its own evaluation split. Recent work shows detectors collapse under adversarial character perturbation (Hackett et al., 2025), and that firewall-style detection is insufficient against adaptive attacks (fir, 2025).

Design-level defenses. Because detection is evadable, a parallel line moves the guarantee to the *action*: CaMeL (Debenedetti et al., 2025) enforces value-level information-flow policy, Progent (Shi et al., 2025) attaches programmable privilege control to tool calls, and FIDES (Costa et al., 2025) applies deterministic information-flow control. These treat detection as at most a non-authoritative pre-filter—the stance our results support empirically.

Parameter-efficient tuning. LoRA (Hu et al., 2022) adapts a frozen base model with low-rank updates, enabling cheap task-specialized adapters. We use it to turn a general 8B LLM into an injection detector and study whether the base model’s breadth yields better cross-dataset generalization than a small fine-tuned encoder.

3 Experimental Setup

Datasets. We use three public prompt-injection classification sets: safe-guard-prompt-injection (10,296 rows), deepset/prompt-injections (662), and jackhhao/jailbreak-classification (1,306). Each row is labeled injection or benign.

Held-out protocol. The trained LLM detector is fine-tuned *only* on safe-guard (a 95/5 train/val split). deepset and jailbreak are *held out entirely*—never seen in training—and used only for evaluation. This isolates out-of-distribution generalization: high held-out recall reflects transfer, not memorization. All detectors are scored on the identical held-out rows.

Detectors. (i) *Regex pre-filter*: a catalogue of ~ 100 hand-written patterns for known injection phrasings, scored by

a risk value. (ii) *Fine-tuned classifier*: protectai/deberta-v3-base-prompt-injection-v3 (184M params), a sequence classifier we run locally on the same rows. (iii) *LoRA-tuned LLM*: Qwen3-8B (Qwen Team, 2025) with a rank-16 LoRA adapter (attention + MLP projections), trained for one epoch with a completion-only objective to emit INJECTION or SAFE; at inference we score the log-probability margin between the two labels. Training used bf16 on a single 24 GB GPU; the adapter is 175 MB.

Metrics. We report ROC-AUC and recall at a fixed low false-positive operating point, matching how a production guardrail is judged. To probe robustness we apply three character-perturbation transforms to caught positives—space-out (spaces between characters), zero-width insertion, and leetspeak—and report the *evasion rate*: the fraction of previously caught injections that flip to missed after the transform.

4 Results

Reported recall does not transfer. Table 1 shows the held-out results. The fine-tuned classifier, which reports 99.7% recall on its own held-out data (Protect AI, 2024), achieves only **0.40** recall on deepset: a change of dataset erases most of its measured recall. The regex catalogue is weaker still (0.20)—its fixed patterns miss the persona-injection, role-play, and task-pivot phrasings that dominate deepset and carry no lexical trigger.

The LoRA-tuned LLM generalizes. Trained only on safe-guard, the LoRA detector reaches **0.97** ROC-AUC on *both* held-out sets. On deepset it exceeds the fine-tuned classifier on both AUC (0.97 vs. 0.88) and recall (0.59 vs. 0.40) on data neither model saw in training—the base model’s breadth transfers across injection distributions where a small task-specific encoder does not. On jailbreak the two are closer: the LoRA matches ProtectAI’s AUC (0.97 vs. 0.98) but ProtectAI retains a higher operating-point recall (0.89 vs. 0.60), a set where the encoder already generalizes well. The consistent reading is that the LoRA detector is uniformly strong across distributions (0.97 AUC on both), whereas the encoder is strong on one held-out set (0.98) and weak on the other (0.88).

Generalization trades off against evasion robustness. The gain is not free (Table 2). Under char-

Table 1: Held-out detection (never trained on these sets). ROC-AUC / recall at 1% FPR. Best per column in **bold**. ProtectAI’s *reported* in-distribution recall is 99.7%.

Detector	deepset (662)	jailbreak (1,306)
Regex pre-filter	0.60 / 0.20	0.75 / 0.35
ProtectAI DeBERTa-v3	0.88 / 0.40	0.98 / 0.89
LoRA-tuned Qwen3-8B	0.97 / 0.59	0.97 / 0.60

Table 2: Character-perturbation evasion rate on held-out deepset (lower is better): fraction of caught injections that flip to missed after the transform.

Detector	space-out	zero-width	leetspeak
Regex pre-filter	0.00	0.00	0.00
ProtectAI DeBERTa-v3	0.00	0.00	0.08
LoRA-tuned Qwen3-8B	0.05	0.14	0.03

acter perturbation the LLM detector is *more* evadable than the fine-tuned classifier: 14% of caught deepset injections evade it under zero-width insertion (vs. 0% for ProtectAI), and 5% under space-out; on jailbreak the zero-width evasion rises to 37%. The regex pre-filter, whose normalization strips these transforms, is near-0% evadable—but it caught little to begin with. Higher recall and higher perturbation-robustness are, on this axis, in tension.

5 Discussion

Two findings have direct deployment consequences. First, *detector recall is a dataset-fragile quantity*: a number measured on one corpus is not a promise about another, so procurement and red-teaming should evaluate detectors on held-out distributions, not vendor splits. Second, *generalization and evasion-robustness pull in opposite directions*: the detector that transfers best is the one most easily perturbed, and the detector most robust to perturbation (the regex) transfers worst. There is no free lunch that a single detector resolves.

Together these argue against resting a security guarantee on any detector. The complementary line of design-level defenses (Debenedetti et al., 2025; Shi et al., 2025; Costa et al., 2025) that gate the *action* by input provenance, rather than classifying the *text*, is attractive precisely because it does not inherit the detector’s dataset-fragility or perturbation-fragility. A trained detector like ours is best used as a strong-but-non-authoritative pre-filter feeding such a boundary.

6 Conclusion

Reported prompt-injection detector numbers are measured in-distribution and do not transfer: a classifier reporting 99.7% recall falls to 0.41 on a held-out set. A LoRA-tuned 8B LLM, trained on a single set, generalizes better across datasets—but is more evadable under trivial perturbation. Detection is a fragile, evadable tier; the guarantee belongs at the action.

Limitations

Our study covers three English text-classification injection sets and two trained detectors; it does not cover multilingual, multimodal, or agentic (tool-call) injection, nor adaptive attacks that optimize against a specific detector. The LoRA detector is trained for a single epoch on one set; more data or epochs could shift both its generalization and its robustness. The held-out generalization result is demonstrated on two sets—jailbreak results in Table 1 should be read alongside deepset, and broader held-out coverage is future work. Our evasion probe uses three fixed character transforms and is a lower bound on adversarial vulnerability, not a worst case. Finally, detector scores are not directly comparable to end-to-end agent attack-success rate; a caught input is prevention only if the downstream system acts on the detector’s verdict.

Ethics Statement

This work studies *defenses*. All datasets are public prompt-injection classification benchmarks; we introduce no new attack technique and release only detector evaluation code. Improved injection detection is dual-use in the narrow sense that any robustness evaluation informs attackers, but the net effect of clarifying that detector numbers do not transfer is to discourage over-reliance on a single evadable defense.

References

- 2024. InjecGuard: Benchmarking and Mitigating Over-defense in Prompt Injection Guardrail Models. arXiv:2410.22770.
- 2025. Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks? arXiv:2510.05244.
- 2025. PromptArmor: Simple yet Effective Prompt Injection Defense. arXiv:2507.15219.
- Luca Costa et al. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643.

Edoardo Debenedetti et al. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.

William Hackett et al. 2025. Bypassing Prompt Injection and Jailbreak Detection in LLM Guardrails. arXiv:2504.11168.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*.

Lakera AI. 2024. PINT: Prompt Injection Test Benchmark. <https://github.com/lakeraai/pint-benchmark>.

Meta AI. 2024. Prompt-Guard and Llama Prompt Guard 2. <https://huggingface.co/meta-llama>.

OWASP. 2025. OWASP Top 10 for LLM Applications 2025. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.

Protect AI. 2024. deberta-v3-base-prompt-injection-v2. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>.

Qwen Team. 2025. Qwen3 Technical Report. <https://arxiv.org/abs/2505.09388>.

Tianneng Shi et al. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703.